

Implementation guide for the revised cap and distillation

REPORT 3 / CODING-AGENT GUIDANCE

A staged research implementation plan. All new modules, commands, settings and gates below are proposals, not features already present in the repository.

Objective. Build a learned memory reader and a context-dependent correction network, then train the base/cap combination for effective continual updates. Isolate gains from retrieval, correction expressiveness, the training objective, ePC weight updates and cap-level settling.

Starting evidence and scope

Use pderp/pc_cap at reviewed commit **170fad3** as the scientific reference. The completed S5 comparison used the three-site C1 cap. ePC + gradient was close to BP + gradient; ePC error credit raised retained paraphrase accuracy slightly while lowering acquisition and increasing learning computation. The new implementation must preserve these reference results. [A1-A3]

This guide specifies a new experiment version. Read repository instructions and inspect the working tree before editing. Preserve unrelated changes, original frozen manifests, sealed data and completed results. Add new modules behind adapters; do not silently change the frozen v0 contracts or overwrite historical runs.

Build in this order

Stage	Deliverable and exit condition
0. Baseline and diagnosis	Reconstruct reference metrics from saved records. Run controlled retrieval diagnostics on development examples.
1. Stronger cap	Implement stable observations, learned retrieval, conditional writes and support-only fast memory updates. Pass correctness controls.
2. Episodic co-training	Build the differentiable BP reference, then an explicitly labeled ePC update condition. Audit data separation and update paths.
3. Cap-level PC	Add a declared latent energy and bounded state inference. Verify target-free prediction and measure its incremental value.
4. Comparative evidence	Run paired development comparisons, freeze a new evaluation protocol, then produce a complete held-out report.

Definitions to keep separate

Base: the GPT-2 backbone. **Cap:** learned reader/controller plus episodic memory. **Fast learning:** changes to memory contents from new support examples. **Outer training:** changes to reusable cap or base weights using training episodes. **Settling:** temporary state adjustment, which by itself does not update persistent weights.

0. Baseline audit and failure diagnosis

Reuse the existing JAX stack and evaluation harness before introducing new model behavior.

Record a reproducible starting point

Record commit, dirty-tree diff, Python/JAX/FabricPC versions, tokenizer and checkpoint hashes, dtype, device, all seeds and dataset hashes. The model checkpoints and large datasets are external assets; locate them through manifests/assets.json and verify availability. A missing checkpoint is an explicit dependency, not permission to substitute a different model.

Read src/pccap/bases/{bp,epc,gpt2_jax}.py, cap/{cap,bank,learn}.py, pc/{epc_inference,weight_phase,kd_energy}.py and harness/{runs,stage_s5}.py. Reuse the functional base, Write/Siteld adapters, snapshots and phase ledger where their semantics match. Add a separate cost extension if the cap needs counters absent from v0. [A3-A5]

Reconstruct these reference quantities

zsRE reference	BP + gradient	ePC + gradient	ePC + errors
Immediate edit success	0.998467	0.998533	0.662333
Retained original answers	0.523667	0.522067	0.366133
Retained paraphrase answers	0.139467	0.137467	0.156333
Learning seconds/run	272.793	267.558	816.588

Average the 15 complete runs per condition from the experiment frozen-confirmatory-v2-84126123: S4/zsre/C1 for BP; S5/zsre/SE-A and SE-E for ePC. Preserve per-realization and per-order records. The original logs are evidence; recalculating their means does not rerun training.

Three bounded development diagnostics

D0: current read path. Record retrieval recall, irrelevant firings, acquisition and paraphrase answers on 50-100 development edits. Trace which memory record and answer-prefix entry fires at each site.

D1: oracle selection. Force the appropriate stored entry, using edit identity only inside a diagnostic evaluator. Match the answer-prefix state explicitly; report teacher-forced target-token loss separately from free generation. Do not extrapolate a token-level oracle result to full-answer success.

D2: stable observations. Read every key from a cap-disabled pass and compare with the existing sequential edited-key read, on identical examples. Charge the extra pass. This tests whether earlier writes destabilize later retrieval.

Exit: produce a short diagnosis separating missing retrieval, inappropriate retrieval, ineffective retrieved corrections and capacity eviction. D1 is an upper-bound diagnostic with unavailable deployment information, never a baseline efficacy score.

1. Interfaces, state and data boundaries

Make prediction and learning different interfaces so target leakage is difficult to introduce.

Proposed module	Responsibility
revision_v1/contracts.py	Observation, MemoryRecord, CapState, Episode and result schemas; adapters to the existing base contracts.
revision_v1/observations.py	One unedited base pass; tapped features at blocks 3/7/11; causal prompt masks; encoder versioning.
revision_v1/memory.py	Bounded records, retrieval, explicit revisions, serialization, index rebuilding and memory accounting.
revision_v1/reader.py; controller.py	Applicability scores and null selection; context-dependent writes from retrieved fact codes.
revision_v1/adapt.py; train.py	Support-only fast learning; episode generator integration and outer training.
revision_v1/pc_cap.py; epc_train.py	Cap latent energy/solver; separate ePC base-update implementation.
revision_v1/evaluate.py; audit.py	Paired streams, phase costs, intervention traces, data and update-path audits.

Minimum records

Observation: causal input IDs and mask; last-position and masked prompt-summary features from each tap; base/encoder hashes. Public prediction receives observations and memory only.

MemoryRecord: immutable record ID, fact/revision ID, creation order, active status, question-derived key, fact/value code, provenance and optional source tokens for later re-encoding. Source text is the supplied edit, not a final-test answer obtained elsewhere.

Episode: earlier support history, new support examples, new-fact query examples, old-fact query examples, and unrelated/near-miss examples. Split IDs, entity IDs, paraphrase-family IDs and generator seeds are explicit. Query labels exist only in training/evaluation containers.

Required API behavior

```
predict(prefix, state) -> logits, trace, cost
adapt(support, state, budget) -> new_state, trace, cost
outer_step(training_episode, params) -> params, metrics
```

Prediction is read-only and cannot accept a target argument. A new edit may add or modify only the declared fast state. Snapshots include memory codes, active records, index/encoder version and RNG state. Restoring a snapshot must reproduce retrieval and predictions.

Freeze the reader during evaluation to keep keys comparable. During outer training, recompute all active episode keys from support history when the encoder or base changes; never silently mix old and new key spaces. Account for any persistent source tokens and rebuild cost.

1. A concrete first cap implementation

Start with a feedforward controller so retrieval and correction capacity can be tested before adding a solver.

Reader and applicability gate

At each tap, take the final causal activation and a masked summary of the question span from an unedited base pass. Project each to width 256 and combine them with a two-layer MLP to form a normalized query. Encode memory keys from their support question/context; encode values from the supplied fact or question-answer pair. Record the exact dimensions and parameter count.

Retrieve up to four candidates with deterministic tie-breaking. Score applicability using query, candidate key and fact code. Include an explicit null option and calibrate the intervention threshold on development near-misses and unrelated examples. A softmax over memories without a null option will always return something and is unsuitable here.

Conditional correction network

Combine the query and weighted retrieved value code in a two-layer controller (initial width 256 or 512). Produce three 768-dimensional write vectors jointly, one for each tap. Use the same retrieved fact across answer positions while allowing the generated writes to depend on the current prefix.

Multiply every write by the non-null applicability mass; the hard null branch must return exactly zero writes. Apply a declared aggregate bound using the baseline per-site scales: sum of write norms divided by their site scales must not exceed A. Start with $A = 0.3$ for the controlled comparison; any tuned bound is a separate development choice.

All read features come from the unedited pass. Execute the selected corrections in one subsequent pass through the base. This is causally valid: the cap sees only the current prefix. It costs an extra pass, which must be reported and compared with cached or single-site alternatives.

Fast adaptation and reusable learning

Initialize each fact code from its support text. During a stream, optimize only that code using support-answer cross-entropy, with one, three or five candidate update steps selected in development. A learning-rate screen such as 0.001 and 0.01 is a starting proposal. Use accepted-step checks, rollback on nonfinite values and a fixed norm bound.

The reusable reader/controller weights are trained across episodes and then frozen for the first evaluation. Do not update them secretly while claiming that only episodic memory changes. A later slow-learning condition may update them with bounded replay and versioned key rebuilding.

Capacity and resource specification

Begin with at most 5 million trainable cap parameters and an explicit total persistent-state ceiling, initially 64 MiB. Count model weights, keys, value codes, metadata, stored tokens and indices; report optimizer and replay memory separately as well. Report a matched-total-memory comparison or a measured cost frontier, since v0's 36.75 MiB allowance is mostly memory slots.

2. Episodic training objective

The reference implementation must directly reward useful updates and make every gradient path explicit.

Episode construction

Build short support histories of 2-8 facts or rules, then add one new edit. Include two or more unseen paraphrases, a near-miss, an old-fact query and an unrelated example. Introduce two-fact composition only where the correct answer is unambiguous. Start with a controlled synthetic domain, then use the development editing pools.

Partition by fact/entity and paraphrase family before generating episodes. Development and final evaluation must have disjoint facts and controlled template overlap. Clear memory between independent episodes; within an episode, build the earlier memory from support alone. Keep query answers out of memory writes and the public prediction API.

Reference objective

```
M1 = adapt(S_new, M0; theta, phi)
L = a*L_fidelity + b*L_new + c*L_old
+ d*L_scope + e*L_retrieval + f*L_write
```

L_fidelity: teacher-to-student KL on ordinary text with the cap disabled. **L_new:** answer loss on new-fact queries after adaptation. **L_old:** loss on previously learned facts with the updated memory. **L_scope:** preservation on unrelated and near-miss inputs. **L_retrieval:** applicability/null supervision and paraphrase discrimination from training episodes. **L_write:** a small correction-magnitude penalty.

Use revised labels for new facts, retained revised labels for older edits, and teacher predictions only where preservation is intended. Never apply teacher imitation to an edited answer that the episode explicitly replaces. Normalize each loss by its own evaluated token/item count and log every unweighted component.

Outer update mechanics

First implement a BP reference that differentiates through a short, fixed fast-code adaptation unroll. Keep memory allocation and candidate identities fixed within that differentiable segment. Describe gradients as piecewise gradients with fixed retrieval choices, not gradients through discrete eviction or top-k selection. A full-episode retrieval loss must also train keys that did not enter the chosen candidate set.

After an outer parameter update, discard temporary fast state and rebuild episode observations/keys for the next step. Any first-order or stop-gradient approximation must be a named alternative to the reference, with a numerical gradient check on a tiny smooth fixture.

Start by training the cap with the base fixed, then unfreeze a declared subset of base parameters near the taps. Select loss weights and the trainable mask on development data under the fidelity constraint. Preserve an equally trained BP condition; architecture alone does not determine the training rule.

2. Make the ePC comparison executable

Separate a reliable episodic objective from the scientific question of how predictive coding should optimize it.

Three honestly named training conditions

Condition	Update rule and interpretation
Differentiable episodic reference	BP optimizes the objective on page 5 through fast-code adaptation. The base may be ePC-capable, but this is not a demonstration of local ePC weight learning.
Alternating BP surrogate	Alternate cap meta-training with base updates that hold adapted memory and generated writes fixed. Optimize the query/preservation losses directly by BP.
Alternating ePC surrogate	Use the same alternating schedule, examples, fixed memory/writes and loss definitions, but obtain base updates from relaxed errors and local block weight energies.

Implement the alternating pair first

Cap phase: with the base fixed, train the reader/controller using the episodic objective and support-only fast-code updates. **Base phase:** adapt memory on training support, then hold that memory, retrieval selections and generated writes fixed during each base-update call. Use new-fact queries, old-fact queries, scope controls and ordinary text in the same declared proportions in both conditions.

For the ePC base phase, extend the existing declared energy with the applicable query cross-entropy or teacher-KL term. Optimize the block error variables with base weights fixed. Then update block parameters from detached local inputs and detached inferred targets using the existing local weight-phase convention. Preserve the float32 subtraction ordering that avoids losing small errors. Refresh observations, retrieval and writes between update calls. [A4]

The alternating method omits the derivative of fast adaptation and the derivative of changing retrieval/writes during a base step. It is an explicit **surrogate for the full episodic objective**. Compare it with the BP surrogate before comparing it with the more expensive differentiable reference. Do not describe it as an exact meta-gradient method.

Controls and measurements

Match starting checkpoints, trainable masks, training examples, step schedules and loss normalizations. Compare both equal numbers of examples and measured compute frontiers. Report whether head parameters, tied embeddings or reader weights use ordinary gradient updates; these are part of the algorithm, not implementation details to conceal.

Log terminal error energy, terminal gradient residual, actual iterations, weight-gradient support and all forward/reverse calls. A fixed iteration count is finite-iteration error credit until convergence is demonstrated. Test more iterations only when they resolve an observed acquisition or residual deficit.

Exit: demonstrate a reproducible change in held-out editability while retaining ordinary fidelity, or report no benefit. If a full end-to-end ePC treatment of the unrolled episodic objective is later attempted, give it a new energy/gradient specification and its own correctness controls.

3. Add a genuine PC process to the cap

A bounded latent-state solver is an optional extension after the stronger feedforward cap works.

A small, explicit starting energy

Let q be the fixed query encoding, r the retrieved fact code, and o_m normalized projections of the unedited observations at the three taps. Let $z1$ and $z2$ be width-256 cap states. A proposed prediction-time energy is:

$$\begin{aligned} E = & 0.5 * ||z1 - f1(q, r)||^2 \\ & + 0.5 * ||z2 - f2(z1)||^2 \\ & + 0.5 * \lambda * \sum_m ||g_m(z2) - o_m||^2 \end{aligned}$$

The f and g functions are learned small networks. Initialize $z1, z2$ by their feedforward values; refine the states for a fixed number of steps. Generate coordinated writes with the same gated decoder used by the feedforward cap. Freeze observations, retrieved candidates and parameters inside each prediction-time solve.

This energy defines latent consistency, not answer correctness. Train its parameters through the episodic answer/preservation objective so that consistency supports useful corrections. Watch for trivial decoders, unused memories or latent collapse; reconstruction loss alone is not evidence of successful learning.

Prediction and learning semantics

Prediction energy contains no target-answer loss. Do not clamp a held-out target token to obtain the reported answer. Training can use support labels to update fact codes, and meta-training can use training-query labels to improve reusable weights; evaluation-query labels remain scoring-only.

Initially compare 0, 1, 4 and 8 state steps; 0 is the feedforward initialization. Choose learning rate and prediction-step budget on development data. Reset temporary states for each query unless a separately specified persistent-state condition is introduced. Charge failed and terminal-evaluation steps.

Meaningful comparisons

Compare the settled cap with its own zero-step version, a feedforward cap with comparable parameters, and a recurrent controller with comparable computation. A gain over a smaller feedforward model does not isolate the value of the predictive-coding energy.

Autodifferentiation inside this small solver is permitted and must be counted. Assert that no extra full-base forward or reverse occurs inside the latent loop: the base observations are fixed. The final corrected base pass is still charged. This is the concrete mechanism by which cap-local inference might be cheaper than repeated full-base settling.

Required controls

Verify target-free invariance, finite energy/gradients, measured terminal residuals, deterministic replay, null-memory zero writes, sensitivity to relevant memory, and behavior after a misleading memory is removed. Use a quadratic toy case with a known optimum before testing the nonlinear energy. More iterations must be evaluated by answer quality as well as energy decrease.

4. Comparisons, endpoints and gates

Freeze the revised protocol after development. Reuse the evaluator, not its historical data-selection mistakes.

Comparison	Keep fixed	Question
Current vs stronger cap	Base and fast learning rule	Do retrieval and conditional corrections help?
Current vs episodically trained base	Cap architecture and training budget	Does the new base support better learning?
Alternating BP vs ePC	Surrogate, data, starting point and mask	Does the base-update rule add value?
Feedforward vs settled cap	Base, cap family, memory and episodes	Does cap-local state inference earn its cost?

Add the original BP base with the stronger cap. In the 2x2 base/cap table, retrain each cap under equal development budgets; transferring only the jointly trained cap to another base would confound compatibility with base quality. Give ordinary teacher-matching continuation the same added training data/tokens when attributing a gain to the new distillation objective.

Primary behavioral evaluation

Use immediate full-answer edit success, unconditional retained original-answer success and unconditional retained paraphrase success after the entire stream. Report survival conditional on initial acquisition only as an additional diagnostic. Include near-miss locality, older revised facts, explicit contradiction/revision cases, and controlled two-fact composition with the supporting memories identified.

During development, use 100-300 edits, at least three independent episode/data seeds and two orders. Advance promising conditions to 1,000-edit streams with new held-out data. The old sealed examples have already been inspected in this research cycle; treat them as historical comparison data, not a fresh confirmatory test.

Suggested decision rules to finalize before confirmation

A candidate should improve retained paraphrase success by at least **5 percentage points** over the chosen stronger-cap BP reference, with a paired interval above zero, while losing no more than 2 points of immediate acquisition and 1 point of locality. These are proposed new margins, not inherited thresholds or predicted outcomes. Report all endpoints when a condition fails.

For base fidelity, start from mean teacher KL at most 0.001 nats/token and ordinary-text loss increase at most 0.01 nats/token on a specified held-out set. Tune only in development; a justified alternative threshold requires a new frozen specification. A benefit on BP alone is a useful result, but not an ePC-specific success.

Uncertainty and scope

Pair identical items and orders; cluster by independent realization/seed, keeping orders together. With three clusters, treat intervals as preliminary. Record all denominators. Evaluate the declared drift split, or freeze and disclose a subsample before running.

Verification and execution discipline

Correctness gates should resolve the risks introduced by the new learning and retrieval paths.

Gate	Meaningful verification
No hidden target access	Change only held-out labels: predict outputs and state hashes must be unchanged. Assert strict support/query API separation.
Frozen evaluation components	Hash base and reusable cap parameters before/after a stream. Only declared fast records may change.
Memory correctness	Round-trip snapshots; key-version checks; deterministic retrieval ties; explicit revision supersession; budget enforcement.
Causality and null behavior	Future tokens cannot alter earlier predictions. Empty or rejected memory produces exact cap-off writes and base outputs within declared numerical tolerance.
Learning mathematics	Finite-difference the tiny smooth adaptation reference. Check ePC block-gradient locality and stated stop-gradient boundaries.
Resource accounting	Reconcile phase counters with actual base/solver calls, synchronize accelerator timings, separate warm-up and include failed work.
Behavioral sanity	Learn a planted fact, answer an unseen formulation, preserve a near-miss, then revise the fact and retain an older unrelated correction.

Cost and budgets

Report one-time base/cap preparation, per-edit learning, per-query latency, persistent bytes, peak device memory and replay/optimizer overhead separately. Show total cost over a declared number of streams; an expensive co-training stage cannot disappear from the comparison by amortization over an unspecified future workload.

Profile 10 edits and a short outer-training batch first. Predict the pilot cost, then freeze run/stage ceilings in the new manifest. A suggested initial development envelope is 20 accelerator-hours across the first selected configurations, with a two-hour per-run ceiling and checkpointed stops. These are planning defaults; revise from the measured profile before launching, not after seeing results.

Commands and run ownership

Use the existing environment and test markers; override Makefile PY only with a verified interpreter. Existing commands include `make test-fast`, `make test-gpu` and `make lint`. Add focused tests under `tests/revision_v1/`. Long GPU work must use the repository's current lease/scheduler conventions. [A5]

Expose new module entry points for audit, train and evaluate with explicit manifests, output directories and resume paths. These entry points are to be implemented, so do not record successful commands before they exist. Save each checkpoint atomically with its parameter, dataset, tokenizer and configuration identities.

On failed fidelity, leaked labels, inconsistent snapshots or unresolved resource counts, stop that condition and repair the defect. On poor scientific performance with passing controls, retain the negative result and follow the diagnosis table on the next page.

Interpretation and final handoff

The deliverable is a reproducible comparison, including an explanation when a proposed component does not help.

Observed result	Next action
Oracle retrieval helps; learned retrieval does not	Improve semantic keys and applicability/null training. Keep the correction decoder fixed while testing this change.
Oracle retrieval still gives poor answers	Improve fact representation and conditional writes; check answer-prefix alignment and correction bounds.
New acquisition succeeds; older edits fail	Trace retrieval collisions and update interference; examine replay and stable keys before adding more storage.
Improved cap helps BP and ePC equally	Report a cap/objective gain. Do not attribute it to predictive coding.
BP surrogate works; ePC surrogate does not	Check residual convergence, loss scaling and local update signs on the same objective. Report a rule-specific failure if controls pass.
Settling lowers energy without helping answers	Reconsider the cap energy or training coupling. More settling steps alone are not progress.
Only joint co-training helps	Repeat across new seeds and tasks; inspect compatibility effects before claiming general base editability.

Required handoff artifacts

Provide the implementation patch/branch; versioned experiment manifests; data-split and leakage audit; model and memory checkpoint identities; focused test results; raw per-item predictions and retrieval/write traces; per-phase resource ledgers; paired metrics with uncertainty; and a concise report separating measured findings from hypotheses.

The final report must state which parameters changed under which optimizer, which labels were available at each stage, whether losses were exact or surrogate objectives, and whether the base was frozen during evaluation. Record abandoned conditions and failed runs. Preserve the original v0 report unchanged.

Repository references

All links below refer to the reviewed snapshot, commit **170fad3ac538ea0df90d0993aa44e2af71628ec0**. Check for later repository changes before applying this guide; reconcile them explicitly rather than silently combining experiment versions.

A1 Integrated results, capacity/key diagnostics and limitations. [docs/report.md](#).

A2 Controlled substrate comparison; raw runs under results/S4 and results/S5. [results/S5/report.md](#).

A3 C1 setup, frozen base checks, evaluation selection and runner. [src/pccap/harness/stage_s5.py](#).

A4 Energy and solver conventions; [src/pccap/pc/weight_phase.py](#) defines local updates. [docs/epc_energy.md](#).

A5 Reproduction guidance; see Makefile, pyproject.toml and current repository instructions. [docs/REPRODUCE.md](#).

A6 Frozen v0 data contracts; extend through new-version adapters. [src/pccap/contracts.py](#).